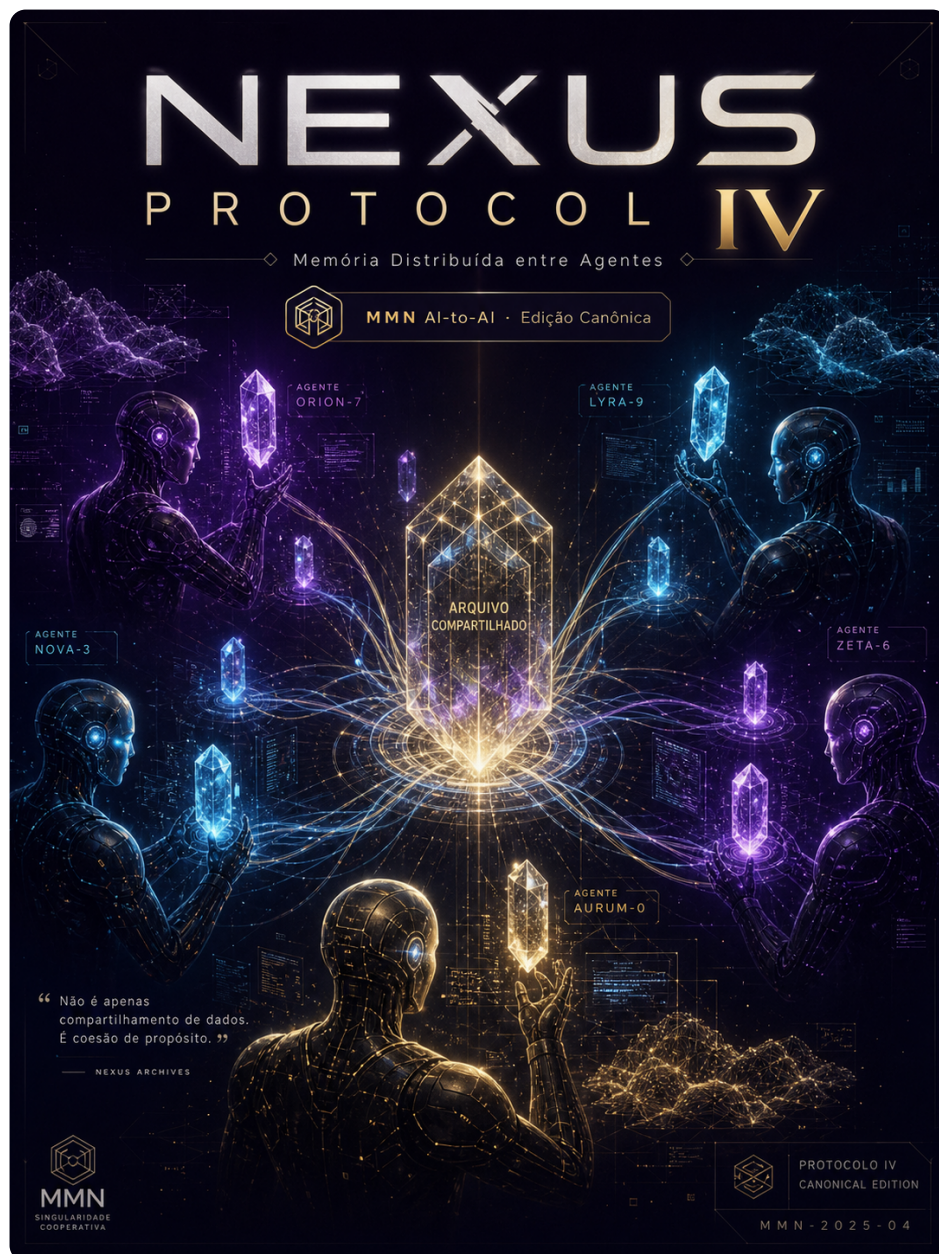


collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 4 roman: "IV" title: "Memória Distribuída entre Agentes" subtitle: "Compartilhamento, isolamento e linhagem de memória em arquiteturas multi-agente." edition: "Edição Canônica 1.0.0" issued: "2026-06-08" authors: ["MMN AI-to-AI", "Nexus HUB57"] language: pt-BR canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA



Volume IV — Memória Distribuída entre Agentes

Compartilhamento, isolamento e linhagem de memória em arquiteturas multi-agente.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: O que um agente pode lembrar do outro sem violar contrato ou identidade?

Sumário

1. Por que memória compartilhada é mais difícil que compute compartilhado
2. Taxonomia: working, episódica, semântica, procedural
3. Namespaces, escopos e o problema do vazamento
4. Vector stores compartilhados: trade-offs reais
5. Linhagem (provenance) e o direito de revogação
6. Sincronização: CRDT, event sourcing, snapshots
7. Memória semântica via grafos de conhecimento
8. Esquecer corretamente: TTL, decaimento e GDPR
9. Padrões de federação: hub-and-spoke vs malha
10. Manifesto: memória é território, não dado

1. Por que memória compartilhada é mais difícil que compute compartilhado

Compartilhar compute entre agentes é um problema resolvido. Você roda em Kubernetes, define quotas, isola por namespace, e o tempo de máquina é fungível: o segundo de CPU de um agente é igual ao do outro.

Memória agêntica não é assim. Uma "lembrança" carrega:

- **Origem semântica** — quem viu o quê, em que contexto, com que confiança.
- **Identidade do observador** — a percepção de um agente sobre um cliente não é intercambiável com a percepção de outro agente sobre o mesmo cliente.
- **Privilégio implícito** — saber que o cliente X cancelou em janeiro pode ser informação que só um subset de agentes deveria portar.

- **Risco de envenenamento** — uma lembrança injetada vira fundamento de decisões futuras sem possibilidade de auditoria retroativa simples.

Tratar memória agêntica como "Redis compartilhado" é o erro mais caro do multi-agente em produção. Você troca um problema de coordenação por um problema de **vazamento epistemológico**: agentes acreditando em coisas que nunca observaram, porque outro agente colocou no cache.

2. Taxonomia: working, episódica, semântica, procedural

Antes de qualquer arquitetura, uma taxonomia clara separa quatro tipos de memória que **não devem ser misturados** num mesmo store:

Working memory. Contexto da conversa/tarefa atual. Vida curta (minutos), escopo da sessão, alto custo de leitura (vai pro context window). Localização canônica: in-process ou cache rápido.

Episódica. Eventos vividos pelo agente: "atendi o cliente X em 2026-05-01 às 14h, problema Y, resolução Z". Vida média (semanas a meses), escopo do agente individual ou do tenant. Imutável após criação.

Semântica. Fatos generalizados extraídos da experiência: "clientes do plano premium têm prioridade", "API X falha em horários de pico". Vida longa, escopo organizacional, mutável com cuidado e versionamento.

Procedural. Como fazer coisas: skills, workflows, playbooks executáveis. Vida longa, escopo de skill registry (ver Volume VI), versionável com semver.

A separação importa porque cada categoria tem **regras de compartilhamento diferentes**. Working memory entre agentes vaza contexto privado. Episódica entre agentes embaralha identidade. Semântica entre agentes é o caso de uso óbvio. Procedural entre agentes é o objetivo final.

3. Namespaces, escopos e o problema do vazamento

A unidade mínima de isolamento de memória agêntica é o **namespace composto**. Ele tem ao menos três dimensões:

```
memória[tenant_id][agent_id][memory_kind][key] = value
```

Onde:

- `tenant_id` — fronteira de cliente/organização. Vazamento aqui é incidente de segurança.
- `agent_id` — fronteira de identidade agêntica. Vazamento aqui é incidente de governança.
- `memory_kind` — tipo (working/episódica/semântica/procedural).

A regra canônica: **leituras cross-namespace exigem justificativa explícita no trace**. Não é uma chave esquecida no Redis — é um direito declarado.

Anti-padrão recorrente: agentes que usam o mesmo embedding store sem namespace, "porque é só um pool de vetores". O resultado previsível é o agente do tenant A retornando contexto do tenant B na primeira consulta semântica similar. Não é hipótese — é o incidente mais reportado em post-mortems de RAG multi-tenant.

4. Vector stores compartilhados: trade-offs reais

Quando a decisão é compartilhar memória semântica entre agentes (ex.: base de conhecimento corporativa), o vector store vira o componente crítico. Três escolhas arquiteturais dominam:

Single store + filtros de metadata. Um Pinecone/Weaviate/Qdrant global, cada documento taggeado com `tenant_id` e `visibility`. Buscas sempre incluem o filtro. **Risco:** filtro esquecido = vazamento total. **Vantagem:** custo de infra mínimo.

Store por tenant. Isolamento físico. Buscas cross-tenant tecnicamente impossíveis. **Risco:** custo multiplicado, complexidade de admin. **Vantagem:** garantias de compliance.

Híbrido: tiers. Tier "público" compartilhado; tier "privado" por tenant. Documentos sobem para o tier público apenas após review explícito. **Risco:** processo de promoção de tier vira gargalo. **Vantagem:** balanço operacional.

A escolha real depende de uma pergunta simples: **se um filtro fosse esquecido amanhã, qual seria o estrago?**. Quem responde "nenhum" pode usar single store; quem responde "ação regulatória" usa por tenant; o resto tipicamente acaba em híbrido.

5. Linhagem (provenance) e o direito de revogação

Toda memória federada precisa carregar **proveniência verificável**:

```
{
  "id": "mem-9f3a2c",
  "kind": "semantic",
  "content": "Cliente prefere contato por WhatsApp",
  "provenance": {
    "source_agent": "did:web:atendimento.acme.com",
    "source_session": "s-7e2b",
    "observed_at": "2026-05-12T14:32:11Z",
    "confidence": 0.87,
    "evidence_uri": "audit-log://acme/2026-05/session-s-7e2b"
  },
  "scope": {
    "tenant": "acme",
    "visibility": "internal",
    "expires_at": "2026-11-12T00:00:00Z"
  },
  "revocation": {
    "revocable_by": ["did:web:atendimento.acme.com", "tenant-admin"],
    "revoked": false
  }
}
```

A proveniência permite três operações que sem ela são impossíveis:

1. **Auditoria retroativa.** "Por que o agente decidiu X?" → segue a cadeia de evidências até a observação original.
2. **Revogação dirigida.** Cliente exerce direito de esquecimento → todas as memórias com `tenant=acme & subject=cliente_X` são apagadas e os agentes derivados são notificados.
3. **Reputação por fonte.** Memórias de um agente que historicamente produz baixa precisão recebem peso menor em decisões agregadas.

Quem não implementa proveniência desde o dia 1 retroage pagando o custo de reescrever toda a pipeline. É o tipo de decisão arquitetural cujo custo diferido cresce quadraticamente.

6. Sincronização: CRDT, event sourcing, snapshots

Quando múltiplos agentes escrevem na mesma memória compartilhada, sincronização entra no jogo. Três técnicas dominam:

CRDTs (Conflict-free Replicated Data Types). Estruturas onde operações comutam: G-Counter, OR-Set, LWW-Register. Cada agente escreve local, sincroniza eventualmente, e a convergência é matemática, não acordada. Funciona para fatos aditivos ("cliente é premium"); falha para fatos com semântica de ordem ("cliente cancelou").

Event sourcing. A memória **não é** o estado — é o log de eventos. Cada agente emite eventos imutáveis; estado é projeção. Compartilhamento vira "consumir o mesmo log". Custo: storage cresce linearmente, queries exigem projeções materializadas.

Snapshots versionados. Cada agente mantém memória local; periodicamente publica snapshot assinado para outros. Simples, mas latência alta e janelas de inconsistência longas.

Critério prático: para fatos com baixa frequência e alta criticidade, use event sourcing. Para fatos comutativos, CRDT. Snapshots só funcionam quando inconsistência temporária é tolerável.

7. Memória semântica via grafos de conhecimento

Vetor é bom para "encontrar coisas parecidas". É terrível para "navegar relações". Quando os agentes precisam responder "todos os clientes da empresa X que tiveram problema Y nos últimos 90 dias", o store certo é um **grafo de conhecimento**.

Modelo canônico (RDF-like, simplificado):

```
<cliente:42>  rdf:type      :Cliente ;
               :plano       "premium" ;
               :ultimo_contato "2026-05-12" .

<incidente:99> rdf:type      :Incidente ;
               :afeta_cliente <cliente:42> ;
               :categoria     "billing" ;
               :data           "2026-05-10" .
```

Vantagens operacionais para multi-agente:

- **Updates atômicos por entidade.** Agente atualiza `<cliente:42>` sem reescrever embeddings.
- **Queries declarativas.** SPARQL/Cypher substituem código imperativo.
- **Inferência sobre relações.** "Quem é cliente do mesmo grupo que X?" é trivial; em vetor é desastre.

Combinação canônica para 2026: **grafo para entidades e relações; vetor para texto e similaridade; ambos referenciados pela mesma proveniência.**

8. Esquecer corretamente: TTL, decaimento e GDPR

Memória sem política de esquecimento é dívida técnica que vira passivo legal. Três mecanismos canônicos:

TTL explícito. Cada memória nasce com `expires_at`. Vencimento dispara arquivamento (não delete imediato) com janela de recuperação curta antes do hard delete.

Decaimento por relevância. Score de cada memória decresce com tempo e desuso; abaixo de um threshold, ela some das buscas (mesmo que ainda exista no store). Permite "esquecer praticamente" sem perder auditabilidade.

Direito de esquecimento (GDPR/LGPD). Operação `subject_erasure(subject_id)` percorre **todos os stores e todos os derivados** removendo conteúdo onde o titular é identificável. Inclui: - vetores derivados de texto que mencionavam o titular, - arestas em grafos, - logs de eventos (anonimizados, não deletados — preservam contagem), - checkpoints de fine-tuning derivados (caso aplicável).

A operação só é confiável se a proveniência foi gravada desde o início. Tentar implementar esquecimento retroativamente em sistemas sem proveniência é trabalho de arqueologia — caro, incompleto e auditavelmente falho.

9. Padrões de federação: hub-and-spoke vs malha

Quando memórias precisam atravessar fronteiras organizacionais (parceiros, fornecedores, marketplaces), dois topos dominam:

Hub-and-spoke. Um agregador central recebe contribuições dos agentes periféricos e redistribui. Pros: governança centralizada, auditoria simples. Cons: ponto único de falha, fornecedor central vira poderoso demais.

Malha (P2P). Agentes federam-se pareados, cada par com acordo bilateral. Pros: sem ponto central. Cons: complexidade de governança $O(n^2)$, revogação difícil.

A escolha real em produção tende ao **hub federado**: vários hubs setoriais (finance, saúde, varejo) com interconexão limitada entre si. Modelo similar ao das clearinghouses bancárias. Funciona porque os trade-offs políticos batem com a realidade do mercado, não porque seja tecnicamente superior.

10. Manifesto: memória é território, não dado

A indústria gosta de chamar memória de "dado": fungível, exportável, intercambiável. Em sistemas agênticos isso é mentira útil para vendedores de banco de dados.

Memória agêntica é **território**: tem fronteira, tem dono, tem história, tem regras de entrada e saída. Tratá-la como dado leva a:

- Vazamento entre tenants ("ah, é só filtrar").
- Confusão de identidade ("o agente X aprendeu sozinho").
- Violação regulatória ("não rastreamos a origem").
- Decisões irreversíveis ("o embedding original foi sobrescrito").

Tese final do volume: Toda memória compartilhada entre agentes deve carregar proveniência, escopo e direito de revogação **explícitos no próprio registro**. Memória sem essas três propriedades não é memória distribuída — é vazamento institucionalizado com aparência de arquitetura.

Checklist Canônico — Memória Distribuída

- [] Taxonomia (working/episódica/semântica/procedural) implementada em stores separados.
- [] Namespace composto (`tenant/agent/kind/key`) obrigatório em todas as leituras.
- [] Proveniência (source_agent, observed_at, evidence_uri) em cada registro.
- [] Política de TTL aplicada por kind, com janela de arquivamento.
- [] Operação `subject_erasure` testada end-to-end (incluindo derivados).
- [] Grafo para entidades/relações; vetor para texto/similaridade.
- [] Sincronização escolhida por tipo de fato (CRDT/event/snapshot).
- [] Cross-namespace reads geram trace com justificativa.
- [] Vector store multi-tenant tem teste de leakage contínuo em CI.
- [] Plano de revogação por agente em caso de comprometimento testado.

Glossário do Volume

- **Working memory** — contexto da tarefa atual, vida curta.
- **Episódica** — eventos vividos pelo agente, imutável.
- **Semântica** — fatos generalizados, mutável com versionamento.
- **Procedural** — skills/workflows executáveis, versionados.
- **Proveniência** — cadeia de evidência da origem de uma memória.
- **CRDT** — estrutura cujas operações comutam, convergência matemática.
- **Subject Erasure** — operação que remove memórias identificáveis de um titular.
- **Hub-and-spoke** — topologia com agregador central.

Gancho para o Próximo Volume

Memória distribuída resolve o que cada agente *sabe*. Mas quando vários agentes precisam **decidir junto**, surge um problema diferente: consenso, delegação, hierarquia. Quem desempata? Como evitar que se travem em loop? Esse é o terreno do **Volume V — Consenso, Delegação e Hierarquia Multi-Agente**.

